Table of Contants

Problem Definition..... 3-4

IMPLEMENTATION DETAILS..... 4-29

Main Class.....4-8

LaserTower Class.....8-10

MissileLauncherTower Class.....10-13

MovingObject Class..... 14-15

Projectile Class……..16-17

SingleShotTower Class…….……18-20

Tower Class……21-22

TripleShotTower Class…….……23-24

Wave Class…….…….…….…25

Test Scenarios…….……26-27

Problem Statement

The goal of this project is to develop a tower defense game using Java and JavaFX, where the player strategically places towers to defend against waves of enemies. The main objective of the game is for the player to protect their base by stopping enemy waves through the strategic placement of towers. The player must place towers at specific points on the map to block the advancement of enemies and win the game by successfully defending against all waves.

During the development of the game, the following core features were implemented:

Level Loading and Map Generation:
The game dynamically generates the map by reading the level configuration from a text file. This file includes information such as path coordinates, wave data, and map dimensions.

Tower Placement:
The player can drag and drop various types of towers (SingleShotTower, TripleShotTower, LaserTower, MissileLauncherTower) onto the map. Each tower type has unique attack mechanics and associated costs.

Enemy Movement:
Enemies follow a predefined path across the map. The player's towers automatically target and attempt to stop these enemies as they move.

Wave System:
Enemies appear in timed waves. Each wave is defined by parameters such as the number of enemies, the spawn interval, and a start delay.

Win and Loss Conditions:

The player wins the level by stopping all enemy waves. However, if enemies reach the base, the player loses lives. The player starts with 5 lives, and the game ends when all lives are lost.

User Interface:
The game features a user interface that displays information such as the player's money, remaining lives, and a countdown to the next wave. It also includes interactive scenes like level selection and game over screens.

Game Loop:
The game runs in a real-time loop, updating enemy movement, tower attacks, and user interactions on every frame.

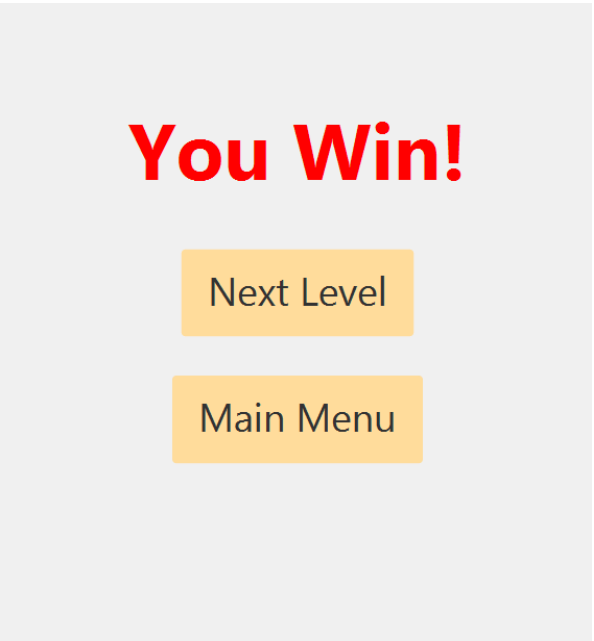
These features offer players a strategic gameplay experience. From a programming perspective, the development process required effective use of object-oriented principles and JavaFX's visual capabilities. For us, the main goal of the project was to learn and grow through building a fully functional game.

IMPLEMENTATION DETAILS

1. Main Class

	Main
	• CELL_SIZE: int • ROWS: int • COLS: int • waveIndex: int • money: int • lives: int • currentLevel: int • nextWaveTimer: Timeline • nextWaveCountdown: double • orangeCells: Set<String> • path: List<int[]> • waves: List<Wave> • objects: List<MovingObject> • towers: List<Tower> • moneyLabel: Label • livesLabel: Label • nextWaveLabel: Label • gameLoop: AnimationTimer • lastFrameTime: long • currentWaveIndex: int • enemiesSpawnedInWave: int • lastSpawnTime: long • waitingForNextWave: boolean • selectedTower: Tower • draggingTower: boolean • previewTower: Tower • waveTimeline: Timeline

	• cellOpacities: Map<String, Double>
-	MovingObject
+	• gameOver(): void • spawnEnemy(): void • checkWinCondition(Stage stage, int currentLevel): void • resetGameState(): void • getCurrentLevel(): int • start(Stage stage): void • showMainMenu(Stage stage): void • showLevelSelect(Stage stage): void • startGame(Stage stage, int level): void • startWaves(): void • startNextWaveCountdown(double seconds): void • updateNextWaveLabel(): void • spawnWave(Wave wave): void • loadLevelFromFile(String filename): void • drawGrid(GraphicsContext gc): void • main(String[] args): void



Description

The Main class serves as the core controller for a tower defense game. It manages the game board, enemies, towers, wave system, and user interface. Built using JavaFX, the game operates on a grid-based board where enemies move along a predefined path, and the player places towers to stop them.

Game Elements and State

Game Board: The game board is defined by the ROWS and COLS variables, creating a grid structure. Each cell has a pixel size determined by CELL_SIZE. The grid is rendered using a Canvas and drawn via the drawGrid method.

Path (path**):** The path that enemies follow is stored as a List<int[]>, where each element represents the row and column coordinates of a cell on the path.

Enemies (objects**):** The MovingObject class represents enemies. Each enemy has:

- A position (posX, posY),
- A speed (speed),
- A health value (health),
- And a path index (currentTargetIndex) to track its progress along the path. Enemies move along the path using the update method and are drawn on the canvas with the draw method. If an enemy reaches the end of the path, the player loses a life (lives).

Towers (towers**):** The player's towers are stored in a List<Tower>. Towers automatically attack enemies and come in different types (e.g., SingleShotTower, LaserTower, TripleShotTower, MissileLauncherTower).

Wave System (waves**):** The List<Wave> stores wave information. Each wave includes parameters such as the number of enemies, spawn interval, and initial delay. The startWaves and spawnWave methods handle the sequential spawning of waves and enemies.

Game State: The player's money (money), lives (lives), and current level (currentLevel) are tracked throughout the game and displayed via UI elements (moneyLabel, livesLabel).

Game Mechanics

Tower Placement: The player places towers by dragging and dropping tower icons from the right side of the screen. Towers have a cost, and placement is only allowed if the player has enough money (money). The canvas.setOnDragOver, canvas.setOnDragDropped, canvas.setOnMousePressed, canvas.setOnMouseDragged, and canvas.setOnMouseReleased event handlers manage tower placement and dragging.

Wave Management: Waves are spawned sequentially using a Timeline. Each wave has a countdown (nextWaveCountdown), which is displayed to the player via

nextWaveLabel. The startNextWaveCountdown and updateNextWaveLabel methods manage this process.

Enemy Movement: Enemies move along the path, and if they reach the end, the player loses a life. If lives drop to zero, the gameOver method is triggered, ending the game.

Win Condition: The player wins when all enemies are defeated and all waves are completed. The checkWinCondition method checks for this and displays the win screen.

User Interface

Main Menu: The showMainMenu method displays a "Start Game" button at the beginning of the game, directing the player to the level selection screen.

Level Selection: The showLevelSelect method creates buttons for levels 1 through 5. When a level is selected, the startGame method is called.

Game Screen: The game screen includes the grid, tower icons, money and lives indicators (moneyLabel, livesLabel), and wave information (nextWaveLabel). The player places towers to stop enemies.

Endgame Screens: When the game ends (gameOver) or the player wins (checkWinCondition), appropriate screens are displayed with buttons like "Main Menu," "Retry," or "Next Level."

Level Loading

The loadLevelFromFile method loads levels from a text file. The file contains the board dimensions (WIDTH, HEIGHT), path coordinates, and wave information. This data is stored in the path and waves lists. Additionally, all cells are initially marked as orange (orangeCells).

Game Loop

The gameLoop (AnimationTimer) manages the main game loop. In each frame:

The screen is cleared (gc.clearRect).
The grid is drawn (drawGrid).
Enemies are updated and drawn.
Towers are updated and drawn.
If a preview tower exists, it is drawn (previewTower).
The win condition is checked (checkWinCondition).

Delta time (deltaTime) is calculated to ensure smooth updates and movements.

Design Decisions and Challenges

During the design of the tower placement mechanic, two approaches were considered:

1. Creating towers first and then updating their positions during dragging.
2. Displaying a preview tower (previewTower) during dragging and creating the actual tower upon dropping.

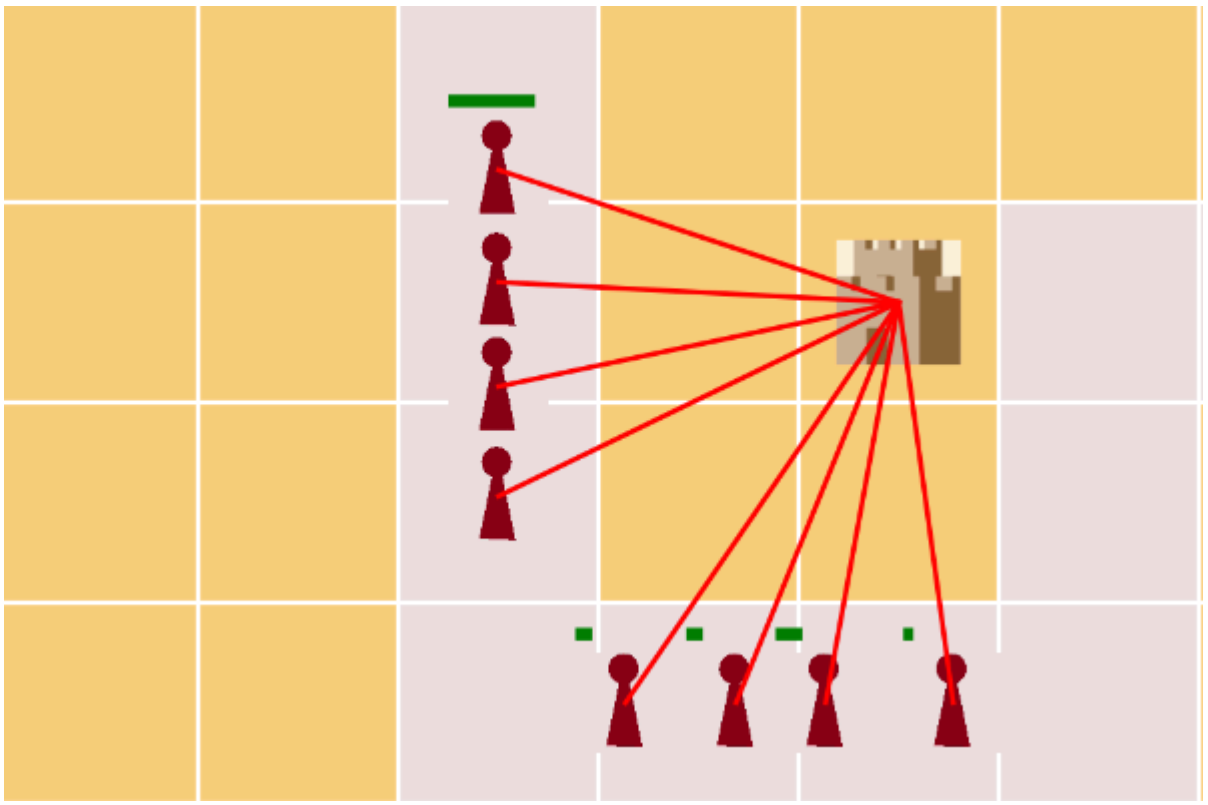
The second approach was chosen because it provides better visual feedback to the player and allows for more controlled placement. The previewTower enables the player to see the tower's range before placing it, reducing placement errors. Additionally, checks in canvas.setOnDragDropped (e.g., ensuring the tower isn't placed on the path or overlapping another tower) enhance the game's consistency.

Conclusion

The Main class successfully orchestrates all core mechanics of the tower defense game. The grid-based board, wave system, tower placement, and enemy movement are effectively implemented using JavaFX tools such as Canvas, Timeline, and AnimationTimer. The user interface provides a clear experience for the player, while the game mechanics offer a strategic and engaging gameplay experience.

2. LaserTower Class

	LaserTower
-	image: Image laserBeams: List<LaserBeam>
-	LaserBeam
+	LaserTower(double x, double y): Constructor update(List<Main.MovingObject> enemies): void draw(GraphicsContext gc): void



Description

The LaserTower class represents a specialized defensive tower in a tower defense game, extending the Tower class. This tower attacks enemies with laser beams and can damage all active enemies within its range simultaneously. Developed using JavaFX, it provides a visual interface for rendering the tower and its laser beams.

Class Elements and State

Image (image**):** The visual representation of the tower, loaded from the /laser_tower.png file and used for rendering the tower.

Laser Beams (laserBeams**):** A List<LaserBeam> that holds the laser beams fired by the tower. This list is cleared and repopulated with new beams for enemies in range during each update cycle.

Inherited Properties: Properties such as x, y (position), range (attack range), and fireRate (firing speed) are inherited from the Tower class. In the LaserTower constructor, range is set to 200 and fireRate to 120.

Inner Class: LaserBeam

The LaserBeam inner class represents a single laser beam with the following fields:

- x1, y1: The starting coordinates of the laser (tower position).
- x2, y2: The ending coordinates of the laser (enemy position).

Method: The draw(GraphicsContext gc) method draws the laser beam as a red line using strokeLine.

Class Mechanics

Update (update**):** Called in each game loop. It damages all active enemies within range (isInRange) by 1 health point each. Multiple enemies can be attacked simultaneously. For each enemy, a LaserBeam object is created and added to the laserBeams list.

Drawing (draw**):** Renders the tower and its laser beams on the screen:

- The tower image is drawn, centered around its position (x - 25, y - 25).
- All laser beams in the laserBeams list are drawn.
- If showRange is true, the attack range is displayed as a red circle using strokeOval.

Design Decisions

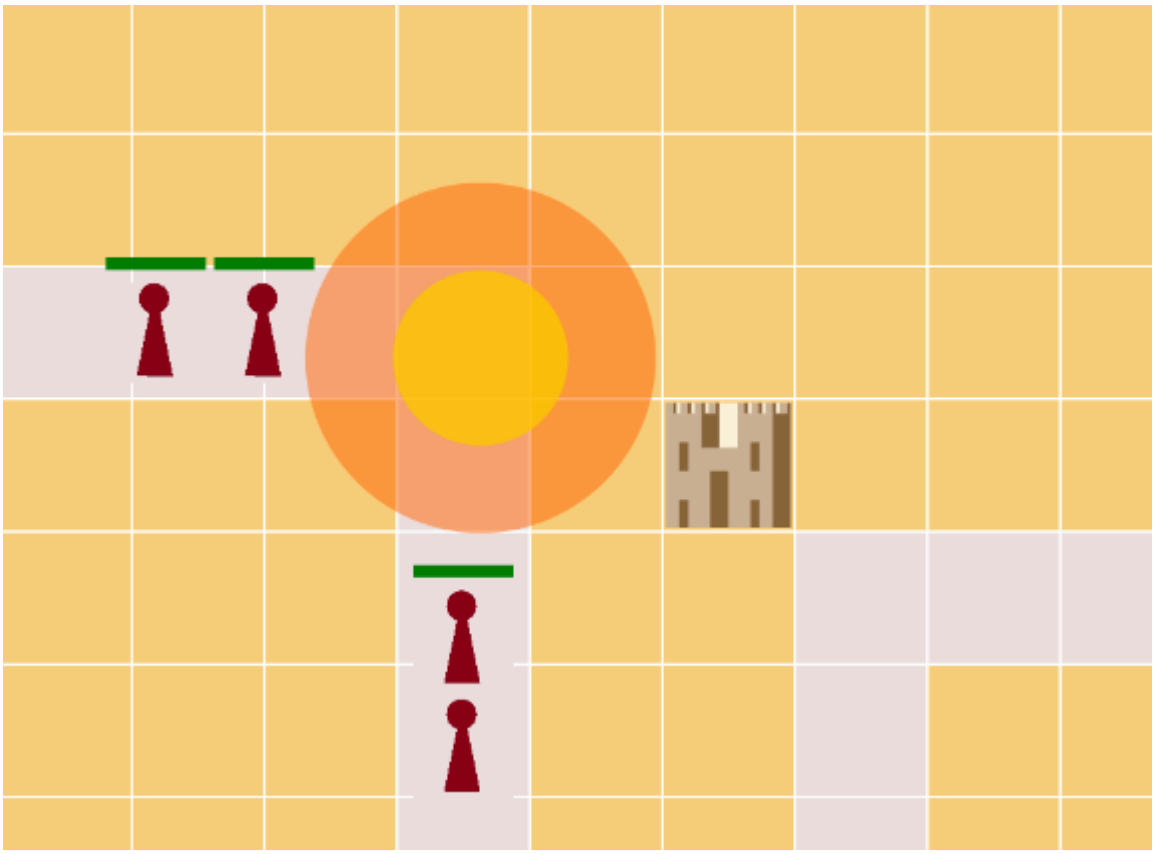
The LaserTower is designed to damage all enemies within its range at once, providing a strategic advantage in the game. Unlike other tower types that typically focus on a single target, the LaserTower can affect multiple enemies simultaneously. The LaserBeam inner class was chosen to represent the visual effect of the laser, offering a modular design and simplifying the rendering of the laser effect.

Conclusion

The LaserTower class functions as an effective defensive unit in the tower defense game. Its ability to damage multiple enemies at once with laser beams offers the player a powerful option against large waves of enemies. The use of JavaFX's graphical tools (GraphicsContext, Image) for rendering the tower and laser beams enhances the game's aesthetic and functional quality

3. MissileLauncherTower

	MissileLauncherTower
	type: int rotationDegree: int xCell: int yCell: int unit Width: double unitHeight: double pane: Pane
	RoadTile(type: int, rotation: int, xCell: int, yCell: int, unitWidth: double, unitHeight: double) getType(): int getRotationDegree(): int getXCell(): int getYCell(): int getUnitWidth(): double getUnitHeight(): double getPane(): Pane setType(type: int): void s etRotationDegree(rotation: int): void setXCell(x: int): void setYCell(y: int): void setUnitWidth(unitWidth: double): void s etUnitHeight(unitHeight: double): void setPane(pane: Pane): void



Detailed Explanation

Overview

The MissileLauncherTower class, extending Tower, represents a tower in the game that fires missiles to deal area-of-effect damage to enemies within its range. With a range of 220 units and a cost of 200, it serves a strategic role, particularly effective against groups of enemies due to its explosive capabilities.

Private Attributes

image: Image: Stores the tower's visual sprite, loaded from "/missile_tower.png".
missiles: List<Missile>: Maintains a list of active missiles launched by the tower.
lastShotTime: long: Records the timestamp of the last missile launch.
fireDelay: long: Defines a 1500ms delay between consecutive missile launches.
explosionRadius: double: Sets the explosion radius to 70 units.
damage: int: Specifies the 35 damage dealt to enemies within the explosion radius.

Public Methods

MissileLauncherTower(double x, double y):

Initializes the tower at the specified (x, y) coordinates with a 220-unit range and 200 cost, loading the tower's image.

update(List<Main.MovingObject> enemies): void:

Updates the tower each frame, checking if the fireDelay has elapsed, firing a missile at the closest enemy if available, and managing missile states.

draw(GraphicsContext gc): void:

- Renders the tower (50x50 pixels, centered at (x, y)) and all active missiles. Draws a red range circle if showRange is true.

Inner Class: Missile

Attributes:

- x, y: double: Current position of the missile.
- targetX, targetY: double: Target coordinates.
- speed: double: Missile speed set to 6.0 units per frame.
- exploded: boolean: Indicates if the missile has exploded.
- explosionTime: long: Records the timestamp of the explosion.

Methods:

- **Missile(double startX, double startY, double endX, double endY)**:
  - Initializes the missile with starting and target coordinates.
- **updatePosition(): void**:
  - Moves the missile toward its target using normalized vector movement, stopping if exploded.
- **reachedTarget(): boolean**:
  - Checks if the missile is within 10 units of its target.
- **explode(): void**:
  - Marks the missile as exploded and sets the explosion timestamp.
- **draw(GraphicsContext gc): void**:
  - Renders the missile as a dark red circle with an orange directional line when moving, or as layered orange/yellow circles for the explosion effect.

Design Notes

The MissileLauncherTower implements a cooldown-based firing system with a 1500ms delay, ensuring balanced gameplay. The inner Missile class handles movement with vector normalization for accurate targeting and includes a 300ms explosion duration for visual feedback. The applyExplosionDamage method efficiently applies 35 damage to all enemies within a 70-unit radius, using an iterator for safe missile removal. This design supports a powerful area-of-effect tower suitable for crowd control in a tower defense game, with potential for further customization (e.g., adjustable explosion radius or damage).

4. MovingObject Class

	MovingObject
-	posX: double posY: double currentTargetIndex: int active: boolean speed: double health: int radius: double
+	MovingObject(speed: double) update(deltaTime: double): void takeDamage(damage: int): voi d draw(gc: GraphicsContext): void

Detailed Explanation

Overview

The MovingObject class models a moving entity, such as an enemy, in a game environment. It manages the object's position, movement, health, and visual representation on a JavaFX canvas, enabling basic enemy behavior in a game.

Public Attributes

posX: double: The x-coordinate of the object's position.
posY: double: The y-coordinate of the object's position.
currentTargetIndex: int: Tracks the index of the target waypoint the object is moving toward.
active: boolean: Indicates if the object is still active (true) or destroyed (false).

speed: double: Defines the movement speed of the object.
health: int: Represents the object's health, starting at 30.
radius: double: The radius (20 units) used for rendering the object as a circle.

Public Methods

MovingObject(speed: double):
- Constructor that initializes the object with a specified speed and default health of 30.

update(deltaTime: double): void:
- Updates the object's position by moving it to the right at a speed adjusted by deltaTime.

takeDamage(damage: int): void:
- Reduces the object's health by the specified damage amount.
- Sets active to false if health drops to 0 or below.

draw(GraphicsContext gc): void:
- Renders the object as a red circle on the canvas if active is true, using the specified radius.

Design Notes

The MovingObject class provides a simple framework for a moving game entity, such as an enemy, with basic movement and health mechanics. The update method currently implements straightforward rightward movement, which can be extended for pathfinding by utilizing currentTargetIndex. The draw method ensures efficient rendering by skipping inactive objects, and the health system allows for interaction with damage-dealing mechanics, such as attacks from towers. This design supports easy integration into a larger game loop while remaining lightweight and extensible.

5. Projectile Class

	Projectile
-	x: double y: double targetX: double

	targetY: double speed: double active: boolean size: double
+	Projectile(x: double, y: double, targetX: double, targetY: double) update(): void draw(gc: GraphicsContext): void isActive(): boolean

Detailed Explanation

Overview

The Projectile class models a projectile, such as a bullet or arrow, in a game environment. It manages the projectile's movement toward a target, tracks its active state, and renders it on a JavaFX canvas with a distinctive appearance.

Public Attributes

x: double: The current x-coordinate of the projectile.
y: double: The current y-coordinate of the projectile.
targetX: double: The x-coordinate of the projectile's target.
targetY: double: The y-coordinate of the projectile's target.
speed: double: The movement speed of the projectile (8 units per frame).
active: boolean: Indicates if the projectile is still active (true until it reaches the target).
size: double: The diameter (6 units) used for rendering the projectile.

Public Methods

Projectile(x: double, y: double, targetX: double, targetY: double):

- Constructor that initializes the projectile with its starting position (x, y) and target coordinates (targetX, targetY).

update(): void:

- Updates the projectile's position by moving it toward the target at the specified speed.
- Deactivates the projectile (active = false) when it is within one speed unit of the target.

draw(GraphicsContext gc): void:

- Renders the projectile as a yellow-filled circle with an orange outline, centered at its current position using the defined size.

isActive(): boolean:

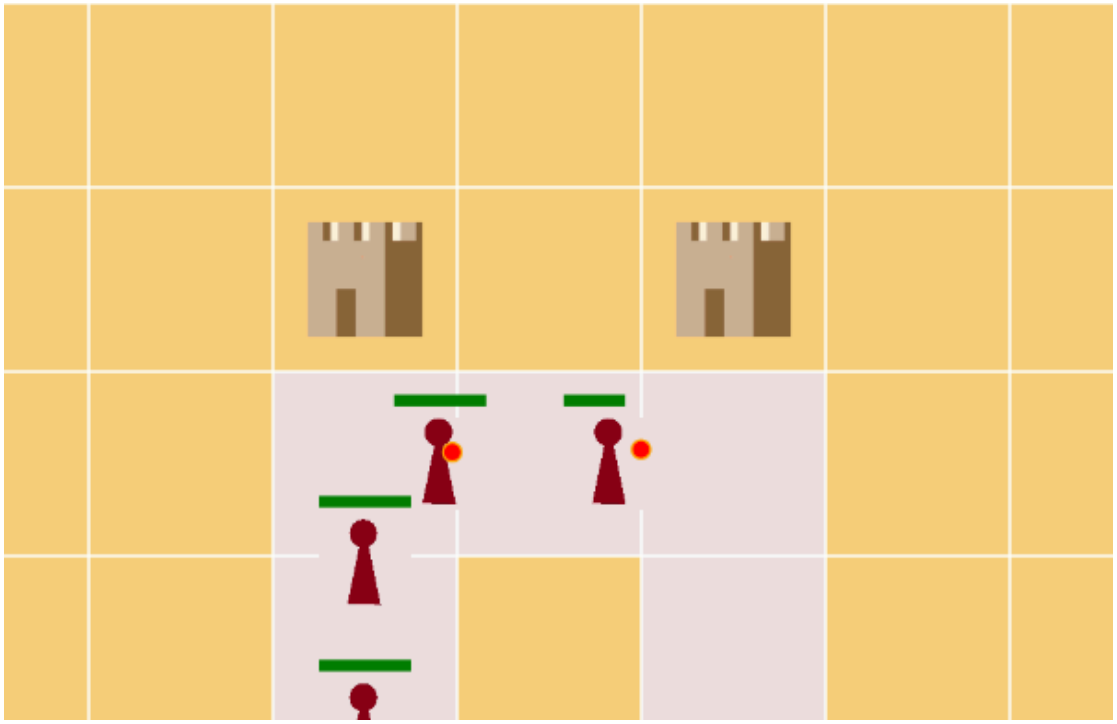
- Returns the active status of the projectile, allowing external checks on its state.

Design Notes

The Projectile class provides a straightforward implementation for a moving projectile with a fixed speed, using vector normalization to ensure consistent movement toward the target. The update method employs a distance-based deactivation mechanism to stop the projectile when it nears its target, preventing overshooting. The draw method uses a dual-color scheme (yellow fill, orange stroke) for visual distinction, and the isActive method supports game logic integration by allowing external management of projectile lifecycles. This design is lightweight and suitable for use in a tower defense or similar game contex

6. SingleShotTower Class

	SingleShotTower
-	image: Image projectiles: List lastShotTime: long fireDelay: long
+	SingleShotTower(x: double, y: double) update(enemies: List): void draw(gc: GraphicsContext): void



Detailed Explanation

Overview

The SingleShotTower class, extending Tower, models a tower in a game that fires single projectiles at the nearest enemy within its range, dealing damage upon impact. It manages projectile creation, movement, and rendering using JavaFX.

Private Attributes

image: Image: Stores the tower's visual sprite, loaded from "/single_tower.png".
projectiles: List<Projectile>: Maintains a list of active projectiles fired by the tower.
lastShotTime: long: Tracks the timestamp of the last shot to enforce the firing delay.
fireDelay: long: Sets a 500ms delay between consecutive shots.

Public Methods

SingleShotTower(x: double, y: double):

- Constructor that initializes the tower at (x, y) with a range of 200 units and a cost of 50.
- Loads the tower's image.

update(List<Main.MovingObject> enemies): void:

- Updates the tower each frame, checking if the firing delay has elapsed.
- Removes inactive projectiles and finds the nearest active enemy in range.
- Fires a new projectile and deals 10 damage to the target if an enemy is found, updating lastShotTime.

draw(GraphicsContext gc): void:

- Renders the tower image (50x50 pixels, centered at (x, y)).
- Updates and draws all active projectiles.
- Draws a red range circle if showRange is true

Inner Class: Projectile

Attributes:

- x, y: double: Current position of the projectile.
- targetX, targetY: double: Target coordinates of the projectile.
- speed: double: Projectile speed (12 units per frame).
- active: boolean: Indicates if the projectile is still active.

Methods:

- **Projectile(x: double, y: double, targetX: double, targetY: double)**:
  - Initializes the projectile with starting and target coordinates.
- **update(): void**:
  - Moves the projectile toward its target, deactivating it when within one speed unit of the target.
- **draw(GraphicsContext gc): void**:
  - Renders the projectile as a red-filled circle with an orange outline (8x8 pixels, centered).

Design Notes

The SingleShotTower implements a cooldown-based firing system with a 500ms delay, ensuring balanced gameplay. The inner Projectile class handles movement with vector normalization for accurate targeting, deactivating upon reaching the target. The tower's damage (10 per shot) and range (200 units) are tuned for a basic offensive role, while the removeIf operation efficiently cleans up inactive projectiles. This design supports integration into a tower defense game with straightforward enemy engagement mechanics

7. Tower Class

	Tower

-	x: double y: double range: double cost: int selected: boolean showRange: boolean
+	Tower(x: double, y: double, range: double, cost: int) distanceTo(enemy: Main.MovingObject): double contains(px: double, py: double): boolean isInRange(enemy: Main.MovingObject): boolean update(enemies: List): void (abstract) draw(gc: GraphicsContext): void (abstract)

Detailed Explanation

Overview

The Tower class is an abstract base class that defines the core properties and behaviors for all tower types in a game. It provides a foundation for tower positioning, range checking, and interaction, with abstract methods for specific tower logic and rendering.

Public Attributes

x: double: The x-coordinate of the tower's center.

y: double: The y-coordinate of the tower's center.
range: double: The tower's effective range (radius of its area of effect).
cost: int: The cost to place the tower in the game.
selected: boolean: Indicates if the tower is currently selected (used for UI interaction).
showRange: boolean: Determines if the tower's range should be visually displayed.

Public Methods

Tower(x: double, y: double, range: double, cost: int):
- Constructor that initializes the tower's position, range, and cost.

distanceTo(enemy: Main.MovingObject): double:
- Calculates the Euclidean distance between the tower and a given enemy.

contains(px: double, py: double): boolean:
- Checks if a point (e.g., mouse click at (px, py)) lies within a 25-unit radius of the tower's center.

isInRange(enemy: Main.MovingObject): boolean:
- Determines if a given enemy is within the tower's range.

update(enemies: List<Main.MovingObject>): void (abstract):
- Abstract method to be implemented by subclasses for updating tower logic (e.g., targeting and firing).

draw(gc: GraphicsContext): void (abstract):
- Abstract method to be implemented by subclasses for rendering the tower on the canvas.

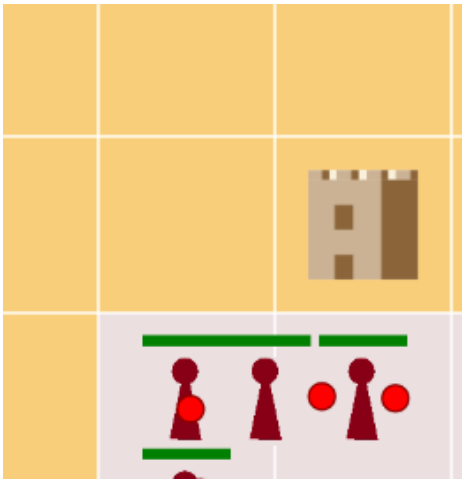
Design Notes

The Tower class serves as a blueprint for all tower types, ensuring consistency in properties like position, range, and cost across different implementations. Its abstract methods (update and draw) enforce that each subclass provides specific behavior and rendering logic, promoting modularity. The distanceTo and isInRange methods provide utility for range-based mechanics, while contains supports user interaction, such as selecting towers via mouse clicks. This design facilitates extensibility for various tower types in a tower defense game.

8. TripleShotTower Class

	TripleShotTower
-	image: Image projectiles: List l astShotTime: long

-	fireDelay: long
+	TripleShotTower(x: double, y: double) update(enemies: List): void draw(gc: GraphicsContext): void



Detailed Explanation

Overview

The TripleShotTower class, extending Tower, models a tower in a game that can target and fire at up to three enemies at once within its range, dealing damage with each shot. It manages projectile creation, movement, and rendering using JavaFX.

Private Attributes

image: Image: Stores the tower's visual sprite, loaded from "/triple_tower.png".
projectiles: List<Projectile>: Maintains a list of active projectiles fired by the tower.
lastShotTime: long: Tracks the timestamp of the last shot to enforce the firing delay.
fireDelay: long: Sets a 700ms delay between consecutive firing cycles.

Public Methods

TripleShotTower(x: double, y: double):

- Constructor that initializes the tower at (x, y) with a range of 200 units and a cost of 150
- Loads the tower's image.

update(List<Main.MovingObject> enemies): void:

- Updates the tower each frame, checking if the firing delay has elapsed.
- Removes inactive projectiles and identifies up to three closest active enemies in range.
- Fires a projectile at each selected enemy, dealing 15 damage, and updates lastShotTime.

draw(GraphicsContext gc): void:

- Renders the tower image (50x50 pixels, centered at (x, y)).
- Updates and draws all active projectiles.
- Draws a red range circle if showRange is true.

Inner Class: Projectile

Attributes:

- x, y: double: Current position of the projectile.
- targetX, targetY: double: Target coordinates of the projectile.
- speed: double: Projectile speed (8 units per frame).
- active: boolean: Indicates if the projectile is still active.

Methods:

- **Projectile(x: double, y: double, targetX: double, targetY: double)**:
  - Initializes the projectile with starting and target coordinates.
- **update(): void**:
  - Moves the projectile toward its target, deactivating it when within one speed unit of the target.
- **draw(GraphicsContext gc): void**:
  - Renders the projectile as a red-filled circle with a dark red outline (12x12 pixels, centered).

Design Notes

The TripleShotTower enhances gameplay by targeting multiple enemies (up to three) simultaneously, using a stream-based approach to efficiently sort and select targets by distance. The 700ms fireDelay balances its rapid multi-target capability, while each projectile deals 15 damage, making it effective against groups. The inner Projectile class ensures consistent movement and rendering, with the removeIf operation maintaining list efficiency by clearing inactive projectiles. This design suits a tower defense game requiring advanced offensive strategies.

9. Wave Class

	Wave

-	enemyCount: int spawnInterval: double delayBeforeStart: double
+	Wave(enemyCount: int, spawnInterval: double, delayBeforeStart: double)

Detailed Explanation

Overview

The Wave class models a single wave of enemies in a game, specifying how many enemies will spawn, the interval between spawns, and the delay before the wave begins. It provides a simple structure for managing enemy waves in a game loop.

Public Attributes

enemyCount: int: The total number of enemies to spawn in this wave.
spawnInterval: double: The time interval (in seconds) between each enemy spawn.
delayBeforeStart: double: The delay (in seconds) before this wave starts spawning enemies.

Public Methods

Wave(enemyCount: int, spawnInterval: double, delayBeforeStart: double):

- Constructor that initializes the wave with the specified enemy count, spawn interval, and delay before starting.

Design Notes

The Wave class is a lightweight data structure for organizing enemy waves in a game, such as in a tower defense scenario. It provides essential parameters (enemyCount, spawnInterval, delayBeforeStart) to control the pacing and difficulty of enemy spawns, allowing for varied gameplay dynamics. Its simplicity ensures easy integration into a game's wave management system, where these attributes can be used to schedule and spawn enemies systematically.

Test Scenorios

In this section, we address some test scenarios encountered during the development of the tower defense game, along with their respective solutions. Below are hypothetical yet realistic cases faced while testing different aspects of the project and the approaches taken to resolve them:

1. **Missile Launcher Tower Overheating Description:**

When the MissileLauncherTower fired too many missiles in rapid succession, the game's performance degraded. This became particularly noticeable during large enemy waves, and the missile explosion effects caused the screen to lag.

Solution: A "heating" mechanism was introduced to dynamically increase the fireDelay time. Additionally, the rendering frequency of explosion effects was reduced (e.g., updated every 2 frames) to optimize performance.

2. **Projectile Collision Detection Failure Description:**

The collision detection mechanism in the Projectile class failed for fast- moving enemies. Although projectiles visually hit the enemies, no damage was applied.

Solution: A more precise distance-checking algorithm was implemented (e.g., using a smaller threshold). The reachedTarget() method was also updated to account for enemy speed dynamically when determining if the projectile reached the target.

3. **TripleShotTower Targeting Inaccuracy Description:**

The TripleShotTower, which targets three enemies within range, sometimes fired multiple projectiles at the same enemy, leading to inefficient damage distribution.

Solution: A check was added in the update method to ensure that each enemy is only targeted once. Enemies were sorted by distance, and unique targets were selected accordingly.

4. **Wave Spawn Timing Issues Description:**

The spawnInterval and delayBeforeStart values in the Wave class were sometimes miscalculated, causing enemies to spawn simultaneously, which destabilized the gameplay.

Solution: A timer was integrated to manage spawn timing. The spawnInterval for each enemy was given a small random variation (e.g., ±0.2 seconds) to create a more natural flow of waves.

5. **MovingObject Health Overflow Description:**

In the MovingObject class, enemy health values increased unexpectedly when they received negative damage (e.g., due to a miscalculation), breaking the game's logic.

Solution: A minimum health check (0 HP) was added to the takeDamage method. Additionally, damage intake was logged for debugging and monitoring purposes.

6. **UI Range Visualization Glitch Description:**

The range circle of towers (when showRange is true) was sometimes drawn with incorrect size or position, making it hard for players to plan their strategy.

Solution: The coordinates of the range circle were recalculated based on the tower's center in the draw method, and setLineWidth was used to fix the stroke thickness, ensuring visual consistency.

7. **Game Freeze on High Enemy Count Description:**

When a large number of enemies (e.g., 50+ MovingObject) were present on screen, the game would occasionally freeze, especially when explosion effects from the MissileLauncherTower were active.

Solution: A cleanup routine was added to immediately remove inactive enemies and projectiles (e.g., exploded missiles) from lists. Additionally, the rendering frequency of explosion effects was reduced.